



Building AI-Driven Interfaces in React Task

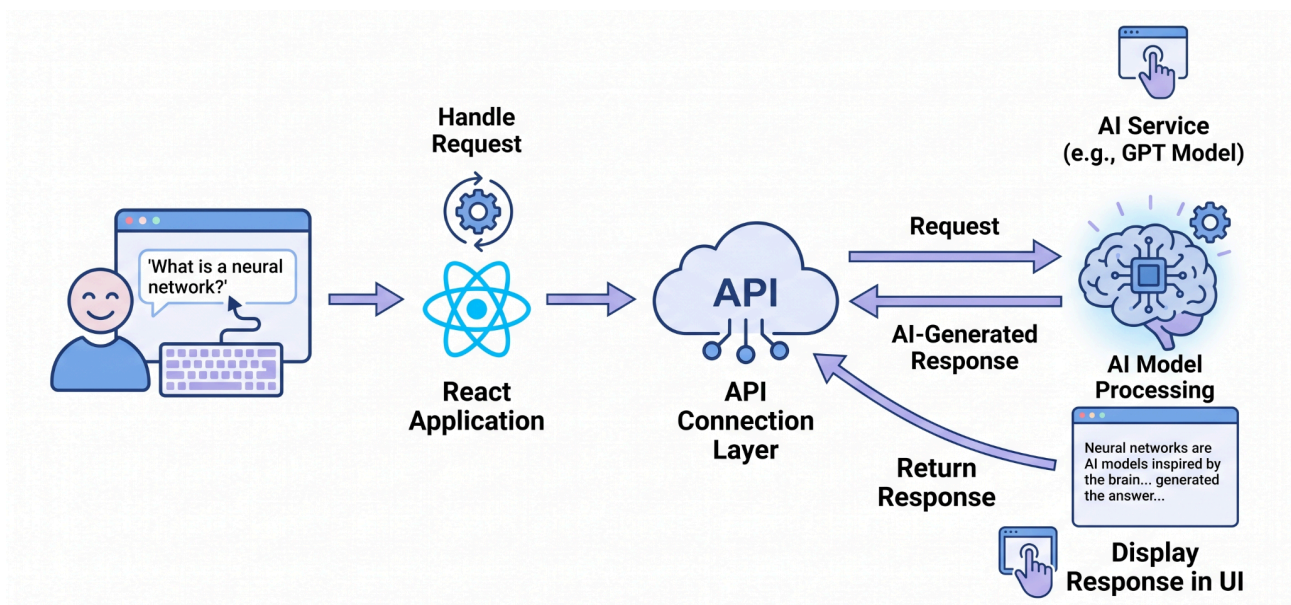
[Visit our website](#)

Introduction

Artificial Intelligence (AI) is becoming an increasingly common feature in modern web applications. Whether you are using a virtual assistant, receiving personalized recommendations, generating content, or interacting with a chatbot, AI is often working behind the scenes to provide a more dynamic and intelligent user experience. However, users rarely interact directly with AI models. Instead, they interact with interfaces that collect information, communicate with AI services, and present the results in a meaningful way. As a front-end developer, your role is not to build the AI model itself but to create the user experience around it. A powerful AI model can quickly become frustrating to use if the interface is poorly designed, confusing, or unresponsive. Conversely, a well-designed interface can make even complex AI capabilities feel intuitive and accessible.

React is particularly well-suited to building AI-powered experiences because it excels at managing dynamic data and updating user interfaces in response to changing application state. Since AI responses are typically generated asynchronously, React's component-based architecture and state management features provide an ideal foundation for building these applications.

In previous lessons, you learned how to build React applications using components and hooks, and how to connect JavaScript applications to external APIs. In this lesson, you will combine these skills to create interfaces that interact with AI systems. Throughout the lesson, you will progressively build an AI Learning Assistant, a simple React application that accepts user questions and displays AI-generated responses.



Note. Created by HyperionDev, 2026.

AI-powered user interfaces

Traditional web applications generally follow predictable interaction patterns. A user submits information, clicks a button, or navigates to a page, and the application responds with predefined content. While the data may vary, the structure of the response is usually known in advance. AI-powered applications work differently. Instead of returning predetermined information, they often generate new content based on the user's request. The exact response cannot always be predicted because it depends on factors such as the user's input, the data available to the AI system, and the model's processing capabilities.

Consider a traditional Frequently Asked Questions (FAQ) page. A user searches for a topic and receives a prewritten answer. In contrast, an AI-powered support assistant can generate a customized response based on the specific question being asked. This flexibility allows AI systems to provide more personalized experiences, but it also introduces additional complexity into the interface. Because AI responses are generated dynamically, developers must design interfaces that can accommodate changing content, loading delays, and occasional errors. The interface must communicate clearly with the user throughout the interaction and provide feedback whenever the system is processing a request.

Several common interface patterns appear repeatedly in AI-powered applications. Conversational interfaces allow users to interact with systems through natural language. Chatbots, virtual tutors, coding assistants, and customer support agents all fall into this category. Recommendation interfaces present suggestions generated by AI systems. These recommendations may include products, courses, movies, articles, or other content that is relevant to the user's interests.

Content generation interfaces allow users to request entirely new content. Examples include AI writing assistants, image generation tools, and code generation platforms. Although these interfaces serve different purposes, they all rely on the same underlying principles: capturing user input, communicating with an AI service, and displaying the generated output.

How React supports AI applications

One of React's greatest strengths is its ability to respond to changes in data. This capability is particularly important when working with AI-powered applications because information is constantly changing throughout the interaction process.

Imagine that a user submits a question to an AI learning assistant. Initially, the application is waiting for user input. Once the question is submitted, the application begins communicating with an AI service. During this period, the interface needs to indicate that processing is taking place. When the AI response arrives, the interface must update automatically to display the new information. Without React, managing these transitions could become complicated. React simplifies this process by allowing developers to store information in state variables and automatically update the interface whenever those values change.

This approach creates applications that feel responsive and interactive. Instead of manually manipulating the Document Object Model (DOM), developers can focus on managing state and allowing React to update the interface accordingly. As a result, React has become one of the most popular frameworks for building AI-powered user experiences.

Managing AI interactions with React state

Before building our AI Learning Assistant, we need to determine what information the application should track. Whenever a user interacts with an AI system, several pieces of information must be managed simultaneously. The application needs to know what the user has typed, whether a request is currently being processed, what response has been received, and whether any errors have occurred. React's `useState()` hook provides a convenient way to manage this information.

```
import { useState } from "react";
function AILearningAssistant() {
  const [prompt, setPrompt] = useState("");
  const [response, setResponse] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  return (
    <div>
      <h1>AI Learning Assistant</h1>
    </div>
  );
}
export default AILearningAssistant;
```

Although this component currently does very little, it establishes the core state structure that many AI applications use. The **prompt** state stores the user's question or instruction. Every time the user types into an input field, this value will be updated. The **response** state stores the information returned by the AI system. When a response is received, React will update the interface automatically.

The **loading** state tracks whether the application is currently waiting for a response. This allows the interface to display feedback while processing takes place.

Finally, the **error** state stores information about any problems that occur during the request. This helps ensure that users receive useful feedback if something goes wrong. Together, these state variables allow the application to manage the complete lifecycle of an AI interaction.

Building the AI Learning Assistant interface

Now that the application's state has been defined, we can begin building the user interface. The first step is to create an input field where users can submit questions.

```
import { useState } from "react";
function AILearningAssistant() {
  const [prompt, setPrompt] = useState("");
  return (
    <div>
      <h1>AI Learning Assistant</h1>
      <input
        type="text"
        value={prompt}
        onChange={(event) =>
          setPrompt(event.target.value)
        }
        placeholder="Ask a question..."
      />
    </div>
  );
}

export default AILearningAssistant;
```

The **value** attribute connects the input field to the **prompt** state variable. This creates what React refers to as a controlled component, meaning that React manages the value displayed in the input field. The **onChange** event updates the state whenever the user types. Each time a character is entered, React stores the updated value inside the **prompt** state variable. At this stage, the application can capture user input, but it

cannot yet do anything with it. To allow users to submit questions, we need to add a button.

Although this appears simple, it represents an important part of the interaction flow. The button acts as the trigger that begins communication with the AI system. Next, we need a place where AI responses can be displayed.

```
<div className="response-panel">
  <h2>Response</h2>
  <p>{response}</p>
</div>
```

This response panel will eventually display the content returned by the AI service. For now, it simply provides a dedicated area within the interface where generated information can appear. By separating the input and output areas of the application, we create a clear and organized user experience.

Creating a response handler

Before connecting to a real AI service, it is useful to simulate how the application will behave when a response is generated. We can do this by creating a simple handler function.

```
const handlePrompt = () => {
  setResponse(
    "React uses components to build reusable user interfaces."
  );
};
```

Next, connect the button to the handler.

```
<button onClick={handlePrompt}>
  Ask AI
</button>
```

When the button is clicked, the response state is updated with a predefined answer. Although this example does not involve real AI processing, it demonstrates a key React concept. Whenever state changes, React automatically re-renders the component and updates the user interface. This behavior forms the foundation of AI-powered applications.

Handling loading states

Unlike the previous example, real AI services do not return responses instantly. Generating a response often takes several seconds depending on the complexity of the request and the capabilities of the AI model. During this time, users need feedback that the system is processing their request. The loading state provides this feedback.

Consider the following example:

```
const handlePrompt = async () => {
  setLoading(true);
  setTimeout(() => {
    setResponse(
      "React uses components to build reusable user interfaces."
    );
    setLoading(false);
  }, 2000);
};
```

In this example, the application simulates a two-second delay before returning a response. While the request is being processed, the loading state remains set to `true`. We can use this value to conditionally display a loading message.

```
{
  loading && (
    <p>Generating response...</p>
  );
}
```

The double ampersand (`&&`) operator is commonly used in React to conditionally render content. When `loading` is `true`, the message is displayed. When it becomes `false`, React automatically removes the message from the interface. Loading indicators are particularly important in AI applications because they reassure users that the system is working as expected.



Spot check 1

Let's see what you can remember from this section.

1. Why are AI-powered interfaces different from traditional user interfaces?
 2. What role does React play in AI-powered applications?
 3. What information is stored in the **prompt**, **response**, **loading**, and **error** state variables?
 4. Why are loading indicators important when interacting with AI services?
 5. How does React update the user interface when state values change?
-

In the next section, we will connect our AI Learning Assistant to an AI service, learn how to send requests using `fetch()`, and explore how recommendation panels can be built using reusable React components.

Connecting the AI Learning Assistant to an AI Service

In the previous section, the AI Learning Assistant displayed a predefined response whenever the user clicked the button. While this allowed us to explore state management and interface updates, it did not involve any actual communication with an AI system. To create a genuine AI-powered application, we need to send requests to an external service and process the responses that are returned. This process follows the same client-server communication principles that you explored in earlier lessons. The main difference is that the server now contains an AI model capable of generating content rather than simply returning stored data.

When a user submits a prompt, the React application sends the request to an AI API. The AI service processes the request and returns a response, which React then displays within the interface. The flow can be summarized as follows:

User Input → React Application → AI Service → React Application → User Interface

Although the process appears straightforward, several steps occur behind the scenes. The application must manage asynchronous communication, handle potential failures, and ensure that the user receives clear feedback throughout the interaction.

Sending requests using fetch()

One of the most common ways to communicate with an external service in React is through the `fetch()` function. This function allows the application to send HTTP requests and process responses asynchronously. The following example demonstrates a simplified request handler for our AI Learning Assistant.

```
const handlePrompt = async () => {
  try {
    setLoading(true);
    setError(null);

    const response = await fetch(
      "https://example-ai-api.com/chat",
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          prompt
        })
      }
    );
    const data = await response.json();
    setResponse(data.response);
  }
  catch (error) {
    setError(
      "Unable to generate a response."
    );
  }
  finally {
    setLoading(false);
  }
};
```

This function performs several important tasks. First, it activates the loading state and clears any previous errors. This ensures that the user receives accurate feedback about the current request. Next, the application sends a POST request to the AI service. The user's prompt is converted into JSON format and included in the request body.

Once a response is returned, the application extracts the generated content and stores it in the `response` state variable. Finally, regardless of whether the request succeeds or fails, the loading state is disabled. This ensures that the interface remains synchronized with the application's actual status. This pattern appears frequently in AI-powered applications and forms the foundation of most AI interactions.

Displaying AI responses

Once the AI service returns a response, the interface must present the information clearly. Rather than displaying the response directly inside the main component, it is often better to create a dedicated component responsible for presenting AI-generated content.

```
function ResponsePanel({ response }) {  
  return (  
    <div className="response-panel">  
      <h2>AI Response</h2>  
      <p>{response}</p>  
    </div>  
  );  
}  
export default ResponsePanel;
```

This approach improves maintainability by separating presentation logic from application logic. The main component focuses on managing state and communication, while the `ResponsePanel` component focuses on displaying information. The component can then be rendered within the application as follows:

```
<ResponsePanel  
  response={response}  
>
```

As the response state changes, React automatically updates the component and displays the latest content.

Building a conversational interface

One of the most common applications of AI is a chatbot. Chatbots allow users to communicate with AI systems through a conversation rather than through individual requests. Unlike the AI Learning Assistant we created earlier, a chatbot must maintain a history of previous messages. This allows users to review earlier interactions and creates a more natural conversational experience. To support this functionality, we need additional state.

```
const [messages, setMessages] = useState([]);
const [prompt, setPrompt] = useState("");
```

The `messages` array stores the conversation history, while the `prompt` state continues to track the user's current input. Whenever a user submits a message, it should be added to the conversation history.

```
const handleSend = () => {
  if (!prompt.trim()) {
    return;
  }
  setMessages([
    ...messages,
    {
      sender: "user",
      text: prompt
    }
  ]);
  setPrompt("");
};
```

The spread operator copies all existing messages before adding the newest message to the array. This ensures that the conversation history grows over time rather than being replaced.

Rendering conversation history

Once messages are stored in state, they must be displayed within the interface. React's `map()` method provides an efficient way to render lists of information.

```
<div className="chat-history">
  {messages.map((message, index) => (
    <p key={index}>
```

```
      {message.sender}: {message.text}
    </p>
  )))}
</div>
```

The `map()` method loops through every message in the array and creates a paragraph element for each one. As additional messages are added to the array, React automatically updates the conversation history displayed on the screen. This dynamic rendering capability is one of the reasons React is so effective for building AI-powered interfaces.

Adding AI responses to the conversation

A chatbot becomes much more useful when AI-generated responses are included alongside user messages. The following example demonstrates how an AI response might be added after a successful request.

```
setMessages((previousMessages) => [
  ...previousMessages,
  {
    sender: "user",
    text: prompt
  },
  {
    sender: "ai",
    text: data.response
  }
]);
```

This approach stores both sides of the conversation within a single state variable. When rendered, the chat history contains a complete record of the interaction between the user and the AI system. Many commercial AI applications, including virtual assistants and customer support tools, rely on this same pattern.

When appending a new interaction to a list array without destroying the questions that came before it, React requires you to use a functional updater pattern inside your state setter. This ensures you are modifying the most up-to-date historical collection snapshot:

```
setHistory((previousHistory) => [
```

```
...previousHistory,  
{  
  question: prompt,  
  answer: data  
}  
]);
```

The spread operator (`...previousHistory`) unpacks all of your existing logs inside the fresh array container before appending the newly completed question-and-answer tracking object onto the end of the collection stream.

Building recommendation panels

Not every AI-powered interface is conversational. Many systems use AI to generate recommendations that help users discover relevant content. Recommendation systems are commonly found in:

- Streaming platforms.
- Online stores.
- Educational platforms.
- News websites.
- Social media applications.

Rather than displaying responses within a conversation, these applications present suggestions in a structured format. Imagine that an AI service returns the following recommendation data:

```
const recommendations = [  
  {  
    id: 1,  
    title: "React Fundamentals",  
    description:  
      "Build a strong foundation in React."  
  },  
  {  
    id: 2,  
    title: "Advanced React",  
    description:  
      "Learn hooks and state management."  
  }  
]
```

```
];
```

To display this information effectively, we can create a reusable recommendation card.

```
function RecommendationCard({
  title,
  description
}) {
  return (
    <div className="card">
      <h3>{title}</h3>
      <p>{description}</p>
    </div>
  );
}
```

This component focuses entirely on presentation. It accepts data through props and displays the information consistently. Next, we create a recommendation panel.

```
function RecommendationPanel({
  recommendations
}) {
  return (
    <div>
      <h2>Recommended Courses</h2>
      {recommendations.map((item) => (
        <RecommendationCard
          key={item.id}
          title={item.title}
          description={item.description}
        />
      ))}
    </div>
  );
}
```

The panel receives an array of recommendations and renders a card for each item. This demonstrates one of React's greatest advantages. A single component can be reused repeatedly to display large amounts of AI-generated content while maintaining a consistent appearance.

Creating reusable AI interface components

As applications grow, placing all functionality inside a single component becomes difficult to maintain. A better approach is to separate functionality into smaller, reusable components. Our AI Learning Assistant could be organized as follows:

- PromptInput
- ResponsePanel
- LoadingIndicator
- ErrorMessage
- RecommendationCard
- RecommendationPanel

Each component has a clearly defined responsibility. The input component collects information from the user. The response panel displays generated content. The loading indicator communicates processing activity, while the error component displays feedback when problems occur. This modular approach improves readability, simplifies testing, and makes future enhancements easier to implement.

To organize your workspace professionally, smaller UI components should be saved in their own dedicated files inside a subfolder named `components`. For these files to communicate, they must use explicit export and import statements.

For example, look at how an individual component declares itself available to your project:

```
// Inside src/components/RecommendationCard.jsx
import React from 'react';

export default function RecommendationCard({ title, description }) {
  return (
    <div className="card">
      <h3>{title}</h3>
      <p>{description}</p>
    </div>
  );
}
```

Your master orchestration component can then consume this block by declaring its relative folder path structure at the top of the file:

```
// Inside src/App.jsx
import React, { useState } from 'react';
import RecommendationCard from '../components/RecommendationCard';
```



Spot check 2

Let's see what you can remember from this section.

1. Why is the `fetch()` function useful when working with AI services?
 2. What role does the `ResponsePanel` component play in the AI Learning Assistant?
 3. Why is conversation history stored in an array when building a chatbot?
 4. How does the `map()` method help when rendering recommendations?
 5. Why is it beneficial to separate AI interfaces into smaller reusable components?
-



Take note

The task(s) below is/are **auto-graded**. An auto-graded task still counts towards your progression and graduation. Give it your best attempt and submit it when you are ready.

When you select “Request Review”, the task is automatically complete, you do not need to wait for it to be reviewed by a mentor.

You will then receive an email with a link to a model answer, as well as an overview of the approach taken to reach this answer.

Take some time to review and compare your work against the model answer. This exercise will help solidify your understanding and provide an opportunity for reflection on how to apply these concepts in future projects.

Once you’ve done that, feel free to progress to the next task.



Auto-graded task

In this activity, you will build a simple AI Learning Assistant using React. Your application will allow a user to type a question, send the question to the Pollinations.AI Text API, and display the generated response in the interface.

The purpose of this activity is not to build an AI model. Instead, you will focus on building the React interface around an AI-powered interaction. You will practise working with state, event handling, asynchronous requests, loading feedback, error handling, and reusable components. For this activity, you will use the **Pollinations.AI Text API**.

API to use

Use the following API endpoint format:

```
https://text.pollinations.ai/YOUR_PROMPT_HERE
```

The user's prompt must be encoded before it is added to the URL. This helps prevent spaces and special characters from breaking the request.

```
const apiUrl =  
  `https://text.pollinations.ai/${encodeURIComponent(prompt)}`;
```

You can then request the AI-generated response using `fetch()`:

```
const response = await fetch(apiUrl);  
const data = await response.text();
```

The API returns plain text, so you should use `response.text()` rather than `response.json()`.

1. Create the React interface

Create a React application called **AI Learning Assistant**. Your interface should include:

- A clear application title.
- A text input or textarea where the user can type a learning-related question.
- A button that sends the question to the API.
- A response area where the AI-generated answer will appear.
- A loading message that appears while the response is being generated.
- An error message area that appears if the request fails.

The interface does not need to be complex, but it should be clear, readable, and easy to use.

2. Store the required state

Use React state to manage the main parts of the interaction. Your component should store the user's prompt, the AI response, the loading status, and any error message. Example:

```
const [prompt, setPrompt] = useState("");
const [response, setResponse] = useState("");
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);
```

These state variables will allow your application to update the interface as the user types, submits a question, waits for a response, and receives the final output.

3. Send the prompt to Pollinations.AI

Create an asynchronous function that sends the user's prompt to the API. Example:

```
const handlePromptSubmit = async () => {
  if (!prompt.trim()) {
    setError("Please enter a question before submitting.");
    return;
  }
  setLoading(true);
  setError(null);
  setResponse("");
  try {
    const apiUrl =
      `https://text.pollinations.ai/${encodeURIComponent(prompt)}`;

    const result = await fetch(apiUrl);
    const data = await result.text();
    setResponse(data);
  } catch (error) {
    setError("Unable to generate a response. Please try again.");
  } finally {
    setLoading(false);
  }
};
```

This function checks that the user has entered a question before sending the request. It then activates the loading state, sends the request, stores the returned text, and disables the loading state once the request is complete.



Take note

⚠️ ****IMPORTANT NOTICE**** ⚠️ The Pollinations legacy text API is being deprecated for ****authenticated users****. Please migrate to our new service at <https://enter.pollinations.ai> for better performance and access to all the latest models. Note: Anonymous requests to text.pollinations.ai are NOT affected and will continue to work normally.

Do not panic if you see this message. For the purposes of this assignment's automated assessment metrics, receiving this text block counts as a **100% successful transaction**. The task requirements evaluate your interface mechanics, not the text payload contents. Your primary goal is to successfully verify that your asynchronous ``fetch`` routine resolves, your ``loading`` indicators toggle, and the returned string maps cleanly into your active UI view and ``history`` panels.

4. Display loading and error feedback

While the API request is running, display a loading message such as:

```
{loading && <p>Generating response...</p>}
```

If an error occurs, display the error message:

```
{error && <p>{error}</p>}
```

This feedback is important because AI responses may not appear instantly. Users should always understand whether the application is working, waiting, or unable to complete the request.

5. Display the AI response

Once the API returns a response, display the answer clearly in the response area. Example:

```
{response && (  
  <div className="response-panel">  
    <h2>AI Response</h2>  
    <p>{response}</p>  
  </div>  
)}
```

The response should be easy to read and visually separate from the input area.

6. Add a conversation history

Extend the application so that previous questions and responses remain visible on the page. Each saved interaction should show the user's question and the AI-generated response. This will make the application feel more like a simple chat or learning assistant.

7. Add a recommendation panel

Create a small recommendation panel that displays suggested learning resources related to React and AI-powered interfaces. You may use mock data for this section.

```
const recommendations = [
```

```
{
  id: 1,
  title: "React State Management",
  description: "Review how useState helps manage changing
interface data."
},
{
  id: 2,
  title: "Async JavaScript",
  description: "Revise how async and await are used when
working with APIs."
}
];
```

Display each recommendation using a reusable component.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Spot check 1 answers

1. Why are AI-powered interfaces different from traditional user interfaces?

AI-powered interfaces generate dynamic responses based on user input, while traditional interfaces typically display predefined information.

2. What role does React play in AI-powered applications?

React manages user input, application state, and interface updates, making it easier to build dynamic AI-powered experiences.

3. What information is stored in the prompt, response, loading, and error state variables?

The `prompt` stores the user's question, the `response` stores the AI-generated answer, the `loading` state tracks whether a request is in progress, and the `error` state stores any error messages.

4. Why are loading indicators important when interacting with AI services?

They inform users that the request is being processed and help prevent confusion while waiting for a response.

5. How does React update the user interface when state values change?

React automatically re-renders components whenever state values are updated.



Spot check 2 answers

1. Why is the `fetch()` function useful when working with AI services?

It allows the application to send requests to an AI API and receive responses.

2. What role does the `ResponsePanel` component play in the AI Learning Assistant?

It displays the AI-generated response to the user.

3. Why is conversation history stored in an array when building a chatbot?

An array allows multiple questions and responses to be stored and displayed over time.

4. How does the `map()` method help when rendering recommendations?

It loops through an array of recommendation data and creates a component for each item.

5. Why is it beneficial to separate AI interfaces into smaller reusable components?

It improves code organization, readability, and maintainability.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
